



Anna Luiza Maneo da Silva

Vinicius de Souza

Daniel Moreira

Kevin Viturino

ATD2 Estrutura de Dados:

Simulador de Estruturas de Dados Lineares em C

Pirassununga – SP

2026

1. Descrição das Estruturas Implementadas

O sistema foi desenvolvido de forma modular em C, implementando 7 estruturas de dados independentes, cada uma restrita a um armazenamento máximo de 100 elementos:

- **Lista Sequencial:** Implementada com vetor estático, exigindo o deslocamento de elementos nas operações de inserção e remoção no meio da lista.
- **Lista Simplesmente Encadeada:** Implementada com uso de ponteiros e alocação dinâmica (malloc/free), permitindo flexibilidade na memória sem necessidade de deslocamento em bloco.
- **Pilha:** Segue o conceito LIFO, controlada unicamente por um índice de topo.
- **Fila:** Segue o conceito FIFO, baseada em vetor estático com índices de início e fim.
- **Fila Circular:** Aperfeiçoamento da fila simples, utilizando aritmética modular ($\% \text{MAX}$) para reaproveitar espaços liberados no início do vetor.
- **Fila Dupla (Deque):** Implementada sobre um vetor circular, permitindo operações flexíveis de inserção e remoção em ambas as extremidades.
- **Fila de Prioridade:** Implementada com busca do elemento de maior valor de prioridade a cada remoção, quebrando o paradigma estrito de tempo de chegada (FIFO).

2. Divisão das Tarefas entre os Membros

- **Anna Maneo:** Codificação, testes e documentação das Listas Sequencial e Encadeada.
- **Daniel Moreira:** Codificação, testes e documentação da Pilha e Fila (simples), configuração e subida do projeto no repositório da disciplina no GitHub, além da resolução de problemas de compilação em ambiente Windows (contorno do utilitário Make via GCC direto).
- **Kevin Viturino:** Codificação, testes e documentação da lógica de índices da Fila Circular e Fila Dupla.
- **Vinicius de Souza:** Estruturação dos menus base (main.c), modularização dos cabeçalhos (.h) e implementação da Fila de Prioridade.

3. Principais Dificuldades

As maiores dificuldades encontradas durante o desenvolvimento envolveram:

- **Lógica Circular:** Compreender e aplicar a aritmética modular correta para o deslocamento dos ponteiros na Fila Circular e na Fila Dupla sem acessar lixo de memória.
- **Ponteiros e Vazamento de Memória:** O controle estrito das alocações da Lista Encadeada exigiu bastante atenção ao implementar funções para limpar a memória (free) ao encerrar o uso, evitando *memory leaks*.
- **Ambiente de Compilação:** Adequar a compilação do projeto modularizado com múltiplos arquivos .c e .h fora de IDEs automatizadas, exigindo a execução manual correta do compilador GCC no terminal.

4. Prints da Execução:

- Lista Simplesmente Encadeada

```
PS C:\Users\danie\Documents\Faculdade\Estrutura de Dados\simulador> .\simulador.exe
SIMULADOR DE ESTRUTURAS DE DADOS
=====
1 - Lista Sequencial
2 - Lista Simplesmente Encadeada
3 - Pilha
4 - Fila
5 - Fila Circular
6 - Fila Dupla
7 - Fila de Prioridade
0 - Sair
=====
Escolha uma estrutura: 2

===== MENU - LISTA SIMPLESMENTE ENCADEADA =====
1 - Inserir no início
2 - Inserir no fim
3 - Remover por ID
4 - Buscar por ID
5 - Exibir lista
6 - Verificar se esta vazia
7 - Quantidade de elementos
0 - Voltar ao menu principal
Opcao: 1
ID do elemento: 1
Dados (texto): Teste
Elemento inserido no início com sucesso.

Pressione ENTER para continuar...

===== MENU - LISTA SIMPLESMENTE ENCADEADA =====
1 - Inserir no início
2 - Inserir no fim
3 - Remover por ID
4 - Buscar por ID
5 - Exibir lista
6 - Verificar se esta vazia
7 - Quantidade de elementos
0 - Voltar ao menu principal
Opcao: 2
ID do elemento: 35
Dados (texto): Teste 2
Elemento inserido no fim com sucesso.

Pressione ENTER para continuar...

===== MENU - LISTA SIMPLESMENTE ENCADEADA =====
1 - Inserir no início
2 - Inserir no fim
3 - Remover por ID
4 - Buscar por ID
5 - Exibir lista
6 - Verificar se esta vazia
7 - Quantidade de elementos
0 - Voltar ao menu principal
Opcao: 5
Lista Encadeada (2 elemento(s)):
INICIO -> [ID:1 | Teste] -> [ID:35 | Teste 2] -> FIM

Pos 1:  [ID: 1 | Dados: Teste | Prioridade: 0]
Pos 2:  [ID: 35 | Dados: Teste 2 | Prioridade: 0]

Pressione ENTER para continuar...[]
```

- Pilha ou Fila

```
===== MENU - PILHA (LIFO) =====
1 - Empilhar (push)
2 - Desempilhar (pop)
3 - Consultar topo (peek)
4 - Exibir pilha
5 - Verificar se esta vazia
6 - Verificar se esta cheia
7 - Quantidade de elementos
0 - Voltar ao menu principal
Opcao: 1
ID do elemento: 5
Dados (texto): Livro 1
Elemento empilhado com sucesso. Topo: 1

Pressione ENTER para continuar...

===== MENU - PILHA (LIFO) =====
1 - Empilhar (push)
2 - Desempilhar (pop)
3 - Consultar topo (peek)
4 - Exibir pilha
5 - Verificar se esta vazia
6 - Verificar se esta cheia
7 - Quantidade de elementos
0 - Voltar ao menu principal
Opcao: 1
ID do elemento: 25
Dados (texto): Livro 2
Elemento empilhado com sucesso. Topo: 2

Pressione ENTER para continuar...

===== MENU - PILHA (LIFO) =====
1 - Empilhar (push)
2 - Desempilhar (pop)
3 - Consultar topo (peek)
4 - Exibir pilha
5 - Verificar se esta vazia
6 - Verificar se esta cheia
7 - Quantidade de elementos
0 - Voltar ao menu principal
Opcao: 1
ID do elemento: 39
Dados (texto): Livro 3
Elemento empilhado com sucesso. Topo: 3

Pressione ENTER para continuar...

===== MENU - PILHA (LIFO) =====
1 - Empilhar (push)
2 - Desempilhar (pop)
3 - Consultar topo (peek)
4 - Exibir pilha
5 - Verificar se esta vazia
6 - Verificar se esta cheia
7 - Quantidade de elementos
0 - Voltar ao menu principal
Opcao: 2
Elemento desempilhado:
[ID: 39 | Dados: Livro 3 | Prioridade: 0]

Pressione ENTER para continuar...

===== MENU - PILHA (LIFO) =====
1 - Empilhar (push)
2 - Desempilhar (pop)
3 - Consultar topo (peek)
4 - Exibir pilha
5 - Verificar se esta vazia
6 - Verificar se esta cheia
7 - Quantidade de elementos
0 - Voltar ao menu principal
Opcao: 4
Pilha (2 elemento(s)) - topo para base:
+-----+
| TOPO |
| [ID: 25 | Dados: Livro 2 | Prioridade: 0] |
| [ID: 5 | Dados: Livro 1 | Prioridade: 0] |
| BASE |
+-----+
```

Pressione ENTER para continuar...[]

- Fila Circular

```
○ Escolha uma estrutura: 5

===== MENU - FILA CIRCULAR =====
1 - Enfileirar
2 - Desenfileirar
3 - Consultar inicio
4 - Consultar fim
5 - Exibir fila circular
6 - Verificar se esta vazia
7 - Verificar se esta cheia
8 - Quantidade de elementos
0 - Voltar ao menu principal
Opcao: 1
ID do elemento: 1
Dados (texto): Teste 1
Elemento enfileirado. fim agora em indice 0.

Pressione ENTER para continuar...

===== MENU - FILA CIRCULAR =====
1 - Enfileirar
2 - Desenfileirar
3 - Consultar inicio
4 - Consultar fim
5 - Exibir fila circular
6 - Verificar se esta vazia
7 - Verificar se esta cheia
8 - Quantidade de elementos
0 - Voltar ao menu principal
Opcao: 1
ID do elemento: 2
Dados (texto): Teste 2
Elemento enfileirado. fim agora em indice 1.

Pressione ENTER para continuar...5

===== MENU - FILA CIRCULAR =====
1 - Enfileirar
2 - Desenfileirar
3 - Consultar inicio
4 - Consultar fim
5 - Exibir fila circular
6 - Verificar se esta vazia
7 - Verificar se esta cheia
8 - Quantidade de elementos
0 - Voltar ao menu principal
Opcao: 5
Fila Circular (2 elemento(s)) [inicio=0, fim=1]:
INICIO -> [ID:1 | Teste 1] -> [ID:2 | Teste 2] (-> INICIO)

Pos 1 (idx 0):  [ID: 1 | Dados: Teste 1          | Prioridade: 0]
Pos 2 (idx 1):  [ID: 2 | Dados: Teste 2          | Prioridade: 0]

Pressione ENTER para continuar...|
```

- Fila de Prioridade e Validação de Erro

```

=====
SIMULADOR DE ESTRUTURAS DE DADOS
=====
1 - Lista Sequencial
2 - Lista Simplesmente Encadeada
3 - Pilha
4 - Fila
5 - Fila Circular
6 - Fila Dupla
7 - Fila de Prioridade
0 - Sair
=====
Escolha uma estrutura: 7

===== MENU - FILA DE PRIORIDADE =====
1 - Inserir elemento com prioridade
2 - Remover elemento de maior prioridade
3 - Consultar proximo a ser removido
4 - Exibir fila ordenada por prioridade
5 - Verificar se esta vazia
6 - Verificar se esta cheia
7 - Quantidade de elementos
0 - Voltar ao menu principal
Opcao: 2
ERRO: fila de prioridade vazia.

Pressione ENTER para continuar...1

===== MENU - FILA DE PRIORIDADE =====
1 - Inserir elemento com prioridade
2 - Remover elemento de maior prioridade
3 - Consultar proximo a ser removido
4 - Exibir fila ordenada por prioridade
5 - Verificar se esta vazia
6 - Verificar se esta cheia
7 - Quantidade de elementos
0 - Voltar ao menu principal
Opcao: 1
ID do elemento: 1
Dados (texto): Teste 1
Prioridade (inteiro, maior = mais urgente): 2
Elemento inserido com prioridade 2.

Pressione ENTER para continuar...

===== MENU - FILA DE PRIORIDADE =====
1 - Inserir elemento com prioridade
2 - Remover elemento de maior prioridade
3 - Consultar proximo a ser removido
4 - Exibir fila ordenada por prioridade
5 - Verificar se esta vazia
6 - Verificar se esta cheia
7 - Quantidade de elementos
0 - Voltar ao menu principal
Opcao: 1
ID do elemento: 2
Dados (texto): Teste 2
Prioridade (inteiro, maior = mais urgente): 1
Elemento inserido com prioridade 1.

Pressione ENTER para continuar...

===== MENU - FILA DE PRIORIDADE =====
1 - Inserir elemento com prioridade
2 - Remover elemento de maior prioridade
3 - Consultar proximo a ser removido
4 - Exibir fila ordenada por prioridade
5 - Verificar se esta vazia
6 - Verificar se esta cheia
7 - Quantidade de elementos
0 - Voltar ao menu principal
Opcao: 4
Fila de Prioridade (2 elemento(s)) - maior prioridade primeiro:
Ordem  Prioridade  Elemento
-----
1       2          [ID: 1 | Dados: Teste 1          | Prioridade: 2]
2       1          [ID: 2 | Dados: Teste 2          | Prioridade: 1]

Pressione ENTER para continuar...

```

5. Conclusão

O desenvolvimento do simulador permitiu ao grupo consolidar, na prática, os conceitos teóricos fundamentais sobre estruturas de dados lineares abordados na disciplina. A implementação em linguagem C exigiu um manuseio rigoroso da memória e da lógica de manipulação de dados, aprofundando nossa compreensão sobre alocação dinâmica (com o uso de ponteiros nas listas encadeadas) e o controle de índices em vetores estáticos (como nas pilhas e filas circulares).

Além disso, a exigência de que cada estrutura mantivesse seus próprios dados de forma independente nos forçou a adotar boas práticas de programação, como a modularização do código e o uso adequado de funções. A divisão estratégica das tarefas garantiu que todos os membros tivessem participação efetiva e dominassem o funcionamento do sistema entregue.

Link repositório Github: <https://github.com/DanielMoreira2023/Simulador-Estruturas-de-Dados>